Task 1 – Understanding Procedural Programming

In this assignment, I will write a payroll processing application using procedural programming paradigm as requested by the client. Before planning and designing the application, it is important to understand what a paradigm is. Therefore, I will explore what a paradigm is, the core concepts of procedural programming, its advantages vs. disadvantages and conclude on why it is best suited for this application.

What is a paradigm?

Programming paradigms are more about how the code is structured and less about which language is used. A programming language can support more than one paradigm. For example, a program written in Javascript can be purely procedural or purely object-oriented.

Some of the most common programming paradigms are Procedural, Object-oriented and Functional programming. Procedural programming is an imperative programming paradigm that involves implementing the behaviour of a program as procedures (or subroutines). Object-oriented, is another imperative programming paradigm but involves creating objects that contain both data structures and associated behaviours. Functional programming is a declarative programming paradigm in which functions are treated like any other data type, bound to identifiers, passed as arguments and returned from other functions.

Core concepts of procedural programming

Top-down approach and sequential execution: Instructions are executed in a sequential order from top to bottom. Some languages allow importing/including libraries and similar resources. For example, in the C programming language, header files can be included in the beginning of the program. These files include pre-defined or user-defined procedures which help the programmer keep the source files tidy and easily readable. Control structures such as (for/while) loops and (if/switch) conditionals are used to manage the flow of the program.

Procedures and modularity: These are also known as functions or subroutines. They are blocks of code, each of which do a specific task and help break down a program into smaller and more manageable modules. These functions can be pre-defined and come with the language libraries. Alternatively, they can be user-defined where the programmer defines what the functions do as per the requirements of the program.

Passing parameters: Functions can take parameters allowing the program to perform operations with different data inputs, increasing the versatility and reusability of the code. The values passed as parameters can be passed by value or by reference. The actual values passed to the functions are called arguments. Parameters and arguments can often be used interchangeably.

Scoping: Local variables lifetime starts and ends within a function. Global variables are declared outside the functions and can be accessed from anywhere in the program. This means functions in one part of the program can change the value of the global variables, which in turn can impact the functionality in the other parts of the program. Some of the programming languages that are predominantly suitable for procedural programming are Fortran, Algol, Cobol, Basic, Pascal and C.

Advantages

Structured approach encourages a clear and logical flow of code execution. It is easy to understand. Especially for beginners.

Easy to test and debug. Modular design makes it easier to isolate and test individual parts of a program.

Performance for straightforward tasks. Procedural code produces efficient programs for specific tasks. It’s faster as there is no overhead for creating objects and memory can be directly manipulated. In OOP, on the other hand, calling a method on an object means, the object's type will be checked at runtime (extra overhead). Procedural functions do not need this step. Functions can be reused in different parts of the program. Especially, when executing batch tasks (such as in payroll transactions) the same routine can be executed as many times as needed.

Disadvantages

Hard to scale for big programs. When the programs get bigger and more complex, it’s more difficult to modify and extend the program or add/change features.

Less secure. There are no access modifiers. Data cannot be encapsulated or hidden from the rest of the program.

It is not suitable to show the relationship between data and behaviour. In OOP, for example, a Car class bundles the vehicle (data) and the drive() method (behaviour) together by design. In procedural programming, drive() function is written separately and car is defined as a variable that is possibly passed to the drive() function. However, there can be other data that can be passed to the drive() function, therefore no limitations as to which data the drive() function can utilize/change.

Code can become difficult to maintain as it grows. If the data structures change, many functions must also be changed.

Best suited program types

Procedural programming is a good fit for small to medium programs, scripting and automation, batch processing (such as payroll calculations), embedded systems and low level programming. In scripting and automation, usually a sequence of steps is executed (such as fill in a form, click a button in UI testing automation or copy files, send a report, call an API etc. in general scripting), there is no need for complex objects or abstractions.

In batch processing, a top-down processing is sufficient and no object relationships are needed. For example, as can be seen in this assignment, the same functions are called sequentially over different employee records to generate a payslip, or display a profile. Tax calculations call the same functions every time with different employee data (i.e salary amount).

In embedded systems, sometimes it's not only convenient but also essential to use procedural programming because embedded systems, such as microcontrollers, have very limited memory and no OS. A procedural language, such as C, can directly access the memory using pointers and produce a compact machine code. This would not be possible with OOP approach.

Conclusion

This application is a payroll processing program aiming to be used in a small-scale business. Therefore, procedural programming paradigm suits this project best as a top-down processing is sufficient to process basic repetitive payroll department processes such as calculating taxes and net pay every month to generate payslip in the same format for each employee.